

Package `org.apache.kafka.clients.producer`

Class `KafkaProducer<K,V>`

`java.lang.Object`[Ⓔ]

`org.apache.kafka.clients.producer.KafkaProducer<K,V>`

All Implemented Interfaces:

`Closeable`[Ⓔ], `AutoCloseable`[Ⓔ], `Producer<K,V>`

```
public class KafkaProducer<K,V>
extends ObjectⒺ
implements Producer<K,V>
```

A Kafka client that publishes records to the Kafka cluster.

The producer is *thread safe* and sharing a single producer instance across threads will generally be faster than having multiple instances.

Here is a simple example of using the producer to send records with strings containing sequential numbers as the key/value pairs.

```
Properties props = new Properties();
props.put("bootstrap.servers", "localhost:9092");
props.put("linger.ms", 1);
props.put("key.serializer", "org.apache.kafka.common.serialization.StringSerializer");
props.put("value.serializer", "org.apache.kafka.common.serialization.StringSerializer");

Producer<String, String> producer = new KafkaProducer<>(props);
for (int i = 0; i < 100; i++)
    producer.send(new ProducerRecord<String, String>("my-topic", Integer.toString(i), Integer.toString(i)));

producer.close();
```

The producer consists of a pool of buffer space that holds records that haven't yet been transmitted to the server as well as a background I/O thread that is responsible for turning these records into requests and transmitting them to the cluster. Failure to close the producer after use will leak these resources.

The `send()` method is asynchronous. When called, it adds the record to a buffer of pending record sends and immediately returns. This allows the producer to batch together individual records for efficiency.

The `acks` config controls the criteria under which requests are considered complete. The default setting "all" will result in blocking on the full commit of the record, the slowest but most durable setting.

If the request fails, the producer can automatically retry. The `retries` setting defaults to `Integer.MAX_VALUE`, and it's recommended to use `delivery.timeout.ms` to control retry behavior, instead of `retries`.

The producer maintains buffers of unsent records for each partition. These buffers are of a size specified by the `batch.size` config. Making this larger can result in more batching, but requires more memory (since we will generally have one of these buffers for each active partition).

By default a buffer is available to send immediately even if there is additional unused space in the buffer. However if you want to reduce the number of requests you can set `linger.ms` to something greater than 0. This will instruct the producer to wait up to that number of milliseconds before sending a request in hope that more records will arrive to fill up the same batch. This is analogous to Nagle's algorithm in TCP. For example, in the code snippet above, likely all 100 records would be sent in a single request since we set our `linger` time to 1 millisecond. However this setting would add 1 millisecond of latency to our request waiting for more records to arrive if we didn't fill up the buffer. Note that records that arrive close together in time will generally batch together even with `linger.ms=0`. So, under heavy load, batching will occur regardless of the `linger` configuration; however setting this to something larger than 0 can lead to fewer, more efficient requests when not under maximal load at the cost of a small amount of latency.

The `buffer.memory` controls the total amount of memory available to the producer for buffering. If records are sent faster than they can be transmitted to the server then this buffer space will be exhausted. When the buffer space is exhausted additional send calls will block. The threshold for time to block is determined by `max.block.ms` after which it returns a failed future with `BufferExhaustedException`.

The `key.serializer` and `value.serializer` instruct how to turn the key and value objects the user provides with their `ProducerRecord` into bytes. You can use the included `ByteArraySerializer` or `StringSerializer` for simple byte or string types.

From Kafka 0.11, the `KafkaProducer` supports two additional modes: the idempotent producer and the transactional producer. The idempotent producer strengthens Kafka's delivery semantics from at least once to exactly once delivery. In particular producer retries will no longer introduce duplicates. The transactional producer allows an application to send messages to multiple partitions (and topics!) atomically.

From Kafka 3.0, the `enable.idempotence` configuration defaults to true. When enabling idempotence, `retries` config will default to `Integer.MAX_VALUE` and the `acks` config will default to `all`. There are no API changes for the idempotent producer, so existing applications will not need to be modified to take advantage of this feature.

To take advantage of the idempotent producer, it is imperative to avoid application level re-sends since these cannot be de-duplicated. As such, if an application enables idempotence, it is recommended to leave the `retries` config unset, as it will be defaulted to `Integer.MAX_VALUE`. Additionally, if a `send(ProducerRecord)` returns an error even with infinite retries (for instance if the message expires in the buffer before being sent), then it is recommended to shut down the producer and check the contents of the last produced message to ensure that it is not duplicated. Finally, the producer can only guarantee idempotence for messages sent within a single session.

To use the transactional producer and the attendant APIs, you must set the `transactional.id` configuration property. If the `transactional.id` is set, idempotence is automatically enabled along with the producer configs which idempotence depends on. Further, topics which are included in transactions should be configured for durability. In particular, the `replication.factor` should be at least 3, and the `min.insync.replicas` for these topics should be set to 2. Finally, in order for transactional guarantees to be realized from end-to-end, the consumers must be configured to read only committed messages as well.

The purpose of the `transactional.id` is to enable transaction recovery across multiple sessions of a single producer instance. It would typically be derived from the shard identifier in a partitioned, stateful, application. As such, it should be unique to each producer instance running within a partitioned application.

All the new transactional APIs are blocking and will throw exceptions on failure. The example below illustrates how the new APIs are meant to be used. It is similar to the example above, except that all 100 messages are part of a single transaction.

```
Properties props = new Properties();
props.put("bootstrap.servers", "localhost:9092");
props.put("transactional.id", "my-transactional-id");
Producer<String, String> producer = new KafkaProducer<>(props, new StringSerializer(), new StringSerializer());

producer.initTransactions();

try {
    producer.beginTransaction();
    for (int i = 0; i < 100; i++)
        producer.send(new ProducerRecord<>("my-topic", Integer.toString(i), Integer.toString(i)));
    producer.commitTransaction();
} catch (ProducerFencedException | OutOfOrderSequenceException | AuthorizationException e) {
    // We can't recover from these exceptions, so our only option is to close the producer and exit.
    producer.close();
} catch (KafkaException e) {
    // For all other exceptions, just abort the transaction and try again.
    producer.abortTransaction();
}
producer.close();
```

As is hinted at in the example, there can be only one open transaction per producer. All messages sent between the `beginTransaction()` and `commitTransaction()` calls will be part of a single transaction. When the `transactional.id` is specified, all messages sent by the producer must be part of a transaction.

The transactional producer uses exceptions to communicate error states. In particular, it is not required to specify callbacks for `producer.send()` or to call `.get()` on the returned `Future`: a `KafkaException` would be thrown if any of the `producer.send()` or transactional calls hit an irrecoverable error during a transaction. See the `send(ProducerRecord)` documentation for more details about detecting errors from a transactional send.

By calling `producer.abortTransaction()` upon receiving a `KafkaException` we can ensure that any successful writes are marked as aborted, hence keeping the transactional guarantees.

This client can communicate with brokers that are version 0.10.0 or newer. Older or newer brokers may not support certain client features. For instance, the transactional APIs need broker versions 0.11.0 or later. You will receive an `UnsupportedVersionException` when invoking an API that is not available in the running broker version.

Field Summary

Fields

Modifier and Type	Field	Description
static final String	NETWORK_THREAD_PREFIX	
static final String	PRODUCER_METRIC_GROUP_NAME	

Constructor Summary

Constructors

Constructor	Description
-------------	-------------

KafkaProducer (Map < String , Object > configs)	A producer is instantiated by providing a set of key-value pairs as configuration.
KafkaProducer (Map < String , Object > configs, Serializer < K > keySerializer, Serializer < V > valueSerializer)	A producer is instantiated by providing a set of key-value pairs as configuration, a key and a value Serializer .
KafkaProducer (Properties properties)	A producer is instantiated by providing a set of key-value pairs as configuration.
KafkaProducer (Properties properties, Serializer < K > keySerializer, Serializer < V > valueSerializer)	A producer is instantiated by providing a set of key-value pairs as configuration, a key and a value Serializer .

Method Summary

All Methods	Instance Methods	Concrete Methods
Modifier and Type	Method	Description
void	abortTransaction()	Aborts the ongoing transaction.
void	beginTransaction()	Should be called before the start of each new transaction.
Uuid	clientIdInstanceId (Duration timeout)	Determines the client's unique client instance ID used for telemetry.
void	close()	Close this producer.
void	close (Duration timeout)	This method waits up to <code>timeout</code> for the producer to complete the sending of all incomplete requests.
void	commitTransaction()	Commits the ongoing transaction.
void	flush()	Invoking this method makes all buffered records immediately available to send (even if <code>linger.ms</code> is greater than 0) and blocks on the completion of the requests associated with these records.
void	initTransactions()	Needs to be called before any other methods when the <code>transactional.id</code> is set in the configuration.
Map < MetricName ,? extends Metric >	metrics()	Get the full set of internal metrics maintained by the producer.
List < PartitionInfo >	partitionsFor (String topic)	Get the partition metadata for the given topic.
void	registerMetricForSubscription (KafkaMetric metric)	Add the provided application metric for subscription.
Future < RecordMetadata >	send (ProducerRecord < K , V > record)	Asynchronously send a record to a topic.
Future < RecordMetadata >	send (ProducerRecord < K , V > record, Callback callback)	Asynchronously send a record to a topic and invoke the provided callback when the send has been acknowledged.
void	sendOffsetsToTransaction (Map < TopicPartition , OffsetAndMetadata > offsets, ConsumerGroupMetadata groupMetadata)	Sends a list of specified offsets to the consumer group coordinator, and also marks those offsets as part of the current transaction.
void	unregisterMetricFromSubscription (KafkaMetric metric)	Remove the provided application metric for subscription.

Methods inherited from class [java.lang.Object](#)

[equals](#), [getClass](#), [hashCode](#), [notify](#), [notifyAll](#), [toString](#), [wait](#), [wait](#), [wait](#)

Field Details

NETWORK_THREAD_PREFIX

```
public static final String↗ NETWORK_THREAD_PREFIX
```

See Also:

[Constant Field Values](#)

PRODUCER_METRIC_GROUP_NAME

```
public static final String↗ PRODUCER_METRIC_GROUP_NAME
```

See Also:

[Constant Field Values](#)

Constructor Details

KafkaProducer

```
public KafkaProducer(Map↗<String↗,Object↗> configs)
```

A producer is instantiated by providing a set of key-value pairs as configuration. Valid configuration strings are documented [here](#)[↗]. Values can be either strings or Objects of the appropriate type (for example a numeric configuration would accept either the string "42" or the integer 42).

Note: after creating a `KafkaProducer` you must always `close()` it to avoid resource leaks.

Parameters:

`configs` - The producer configs

KafkaProducer

```
public KafkaProducer(Map↗<String↗,Object↗> configs,  
                    Serializer<K> keySerializer,  
                    Serializer<V> valueSerializer)
```

A producer is instantiated by providing a set of key-value pairs as configuration, a key and a value `Serializer`. Valid configuration strings are documented [here](#)[↗]. Values can be either strings or Objects of the appropriate type (for example a numeric configuration would accept either the string "42" or the integer 42).

Note: after creating a `KafkaProducer` you must always `close()` it to avoid resource leaks.

Parameters:

`configs` - The producer configs

`keySerializer` - The serializer for key that implements `Serializer`. The `configure()` method won't be called in the producer when the serializer is passed in directly.

`valueSerializer` - The serializer for value that implements `Serializer`. The `configure()` method won't be called in the producer when the serializer is passed in directly.

KafkaProducer

```
public KafkaProducer(Properties↗ properties)
```

A producer is instantiated by providing a set of key-value pairs as configuration. Valid configuration strings are documented [here](#)[↗].

Note: after creating a `KafkaProducer` you must always `close()` it to avoid resource leaks.

Parameters:

`properties` - The producer configs

KafkaProducer

```
public KafkaProducer(Properties🔗 properties,
                    Serializer<K> keySerializer,
                    Serializer<V> valueSerializer)
```

A producer is instantiated by providing a set of key-value pairs as configuration, a key and a value `Serializer`. Valid configuration strings are documented [here](#)[🔗].

Note: after creating a `KafkaProducer` you must always `close()` it to avoid resource leaks.

Parameters:

`properties` - The producer configs

`keySerializer` - The serializer for key that implements `Serializer`. The `configure()` method won't be called in the producer when the serializer is passed in directly.

`valueSerializer` - The serializer for value that implements `Serializer`. The `configure()` method won't be called in the producer when the serializer is passed in directly.

Method Details

initTransactions

```
public void initTransactions()
```

Needs to be called before any other methods when the `transactional.id` is set in the configuration. This method does the following:

1. Ensures any transactions initiated by previous instances of the producer with the same `transactional.id` are completed. If the previous instance had failed with a transaction in progress, it will be aborted. If the last transaction had begun completion, but not yet finished, this method awaits its completion.
2. Gets the internal producer id and epoch, used in all future transactional messages issued by the producer.

Note that this method will raise **`TimeoutException`** if the transactional state cannot be initialized before expiration of `max.block.ms`. Additionally, it will raise **`InterruptedException`** if interrupted. It is safe to retry in either case, but once the transactional state has been successfully initialized, this method should no longer be used.

Specified by:

`initTransactions` in interface `Producer<K,V>`

Throws:

`IllegalStateException`[🔗] - if no `transactional.id` has been configured

`UnsupportedVersionException` - fatal error indicating the broker does not support transactions (i.e. if its version is lower than 0.11.0.0)

`AuthorizationException` - error indicating that the configured `transactional.id` is not authorized, or the idempotent producer id is unavailable. See the exception for more details. User may retry this function call after fixing the permission.

`KafkaException` - if the producer has encountered a previous fatal error or for any other unexpected error

`TimeoutException` - if the time taken for initialize the transaction has surpassed `max.block.ms`.

`InterruptedException` - if the thread is interrupted while blocked

beginTransaction

```
public void beginTransaction()
        throws ProducerFencedException
```

Should be called before the start of each new transaction. Note that prior to the first invocation of this method, you must invoke `initTransactions()` exactly one time.

Specified by:

`beginTransaction` in interface `Producer<K,V>`

Throws:

`IllegalStateException`[🔗] - if no `transactional.id` has been configured or if `initTransactions()` has not yet been invoked

`ProducerFencedException` - if another producer with the same `transactional.id` is active

`InvalidProducerEpochException` - if the producer has attempted to produce with an old epoch to the partition leader. See the exception for more details

`UnsupportedVersionException` - fatal error indicating the broker does not support transactions (i.e. if its version is lower than 0.11.0.0)

`AuthorizationException` - fatal error indicating that the configured `transactional.id` is not authorized. See the exception for more details

`KafkaException` - if the producer has encountered a previous fatal error or for any other unexpected error

sendOffsetsToTransaction

```
public void sendOffsetsToTransaction(Map<TopicPartition, OffsetAndMetadata> offsets,
                                     ConsumerGroupMetadata groupMetadata)
    throws ProducerFencedException
```

Sends a list of specified offsets to the consumer group coordinator, and also marks those offsets as part of the current transaction. These offsets will be considered committed only if the transaction is committed successfully. The committed offset should be the next message your application will consume, i.e. `nextRecordToBeProcessed.offset()` (or `ConsumerRecords.nextOffsets()`). You should also add the leader epoch as commit metadata, which can be obtained from `ConsumerRecord.leaderEpoch()` or `ConsumerRecords.nextOffsets()`.

This method should be used when you need to batch consumed and produced messages together, typically in a consume-transform-produce pattern. Thus, the specified `groupMetadata` should be extracted from the used `consumer` via `KafkaConsumer.groupMetadata()` to leverage consumer group metadata. This will provide stronger fencing than just supplying the `consumerGroupId` and passing in `new ConsumerGroupMetadata(consumerGroupId)`, however note that the full set of consumer group metadata returned by `KafkaConsumer.groupMetadata()` requires the brokers to be on version 2.5 or newer to understand.

This method is a blocking call that waits until the request has been received and acknowledged by the consumer group coordinator; but the offsets are not considered as committed until the transaction itself is successfully committed later (via the `commitTransaction()` call).

Note, that the consumer should have `enable.auto.commit=false` and should also not commit offsets manually (via `sync` or `async` commits). This method will raise **TimeoutException** if the producer cannot send offsets before expiration of `max.block.ms`. Additionally, it will raise **InterruptedException** if interrupted.

Specified by:

`sendOffsetsToTransaction` in interface `Producer<K,V>`

Throws:

IllegalStateException - if no `transactional.id` has been configured or no transaction has been started.

ProducerFencedException - fatal error indicating another producer with the same `transactional.id` is active

UnsupportedVersionException - fatal error indicating the broker does not support transactions (i.e. if its version is lower than 0.11.0.0) or the broker doesn't support the latest version of transactional API with all consumer group metadata (i.e. if its version is lower than 2.5.0).

UnsupportedForMessageFormatException - fatal error indicating the message format used for the offsets topic on the broker does not support transactions

AuthorizationException - fatal error indicating that the configured `transactional.id` is not authorized, or the consumer group id is not authorized.

CommitFailedException - if the commit failed and cannot be retried (e.g. if the consumer has been kicked out of the group). Users should handle this by aborting the transaction.

FencedInstanceIdException - if this producer instance gets fenced by broker due to a mis-configured consumer instance id within group metadata.

InvalidProducerEpochException - if the producer has attempted to produce with an old epoch to the partition leader. See the exception for more details

KafkaException - if the producer has encountered a previous fatal or abortable error, or for any other unexpected error

TimeoutException - if the time taken for sending the offsets has surpassed `max.block.ms`.

InterruptedException - if the thread is interrupted while blocked

commitTransaction

```
public void commitTransaction()
    throws ProducerFencedException
```

Commits the ongoing transaction. This method will flush any unsent records before actually committing the transaction.

Further, if any of the `send(ProducerRecord)` calls which were part of the transaction hit irrecoverable errors, this method will throw the last received exception immediately and the transaction will not be committed. So all `send(ProducerRecord)` calls in a transaction must succeed in order for this method to succeed.

If the transaction is committed successfully and this method returns without throwing an exception, it is guaranteed that all **callbacks** for records in the transaction will have been invoked and completed. Note that exceptions thrown by callbacks are ignored; the producer proceeds to commit the transaction in any case.

Note that this method will raise **TimeoutException** if the transaction cannot be committed before expiration of `max.block.ms`, but this does not mean the request did not actually reach the broker. In fact, it only indicates that we cannot get the acknowledgement response in time, so it's up to the application's logic to decide how to handle timeouts. Additionally, it will raise **InterruptedException** if interrupted. It is safe to retry in either case, but it is not possible to attempt a different operation (such as `abortTransaction`) since the commit may already be in the progress of completing. If not retrying, the only option is to close the producer.

Specified by:

`commitTransaction` in interface `Producer<K,V>`

Throws:

`IllegalStateException` - if no `transactional.id` has been configured or no transaction has been started

`ProducerFencedException` - fatal error indicating another producer with the same `transactional.id` is active

`UnsupportedVersionException` - fatal error indicating the broker does not support transactions (i.e. if its version is lower than 0.11.0.0)

`AuthorizationException` - fatal error indicating that the configured `transactional.id` is not authorized. See the exception for more details

`InvalidProducerEpochException` - if the producer has attempted to produce with an old epoch to the partition leader. See the exception for more details

`KafkaException` - if the producer has encountered a previous fatal or abortable error, or for any other unexpected error

`TimeoutException` - if the time taken for committing the transaction has surpassed `max.block.ms`.

`InterruptedException` - if the thread is interrupted while blocked

abortTransaction

```
public void abortTransaction()  
           throws ProducerFencedException
```

Aborts the ongoing transaction. Any unflushed produce messages will be aborted when this call is made. This call will throw an exception immediately if any prior `send(ProducerRecord)` calls failed with a `ProducerFencedException` or an instance of `AuthorizationException`.

Note that this method will raise `TimeoutException` if the transaction cannot be aborted before expiration of `max.block.ms`, but this does not mean the request did not actually reach the broker. In fact, it only indicates that we cannot get the acknowledgement response in time, so it's up to the application's logic to decide how to handle timeouts. Additionally, it will raise `InterruptedException` if interrupted. It is safe to retry in either case, but it is not possible to attempt a different operation (such as `commitTransaction()`) since the abort may already be in the progress of completing. If not retrying, the only option is to close the producer.

Specified by:

`abortTransaction` in interface `Producer<K,V>`

Throws:

`IllegalStateException` - if no `transactional.id` has been configured or no transaction has been started

`ProducerFencedException` - fatal error indicating another producer with the same `transactional.id` is active

`InvalidProducerEpochException` - if the producer has attempted to produce with an old epoch to the partition leader. See the exception for more details

`UnsupportedVersionException` - fatal error indicating the broker does not support transactions (i.e. if its version is lower than 0.11.0.0)

`AuthorizationException` - fatal error indicating that the configured `transactional.id` is not authorized. See the exception for more details

`KafkaException` - if the producer has encountered a previous fatal error or for any other unexpected error

`TimeoutException` - if the time taken for aborting the transaction has surpassed `max.block.ms`.

`InterruptedException` - if the thread is interrupted while blocked

send

```
public Future<RecordMetadata> send(ProducerRecord<K,V> record)
```

Asynchronously send a record to a topic. Equivalent to `send(record, null)`. See `send(ProducerRecord, Callback)` for details.

Specified by:

`send` in interface `Producer<K,V>`

send

```
public Future<RecordMetadata> send(ProducerRecord<K,V> record,  
                                   Callback callback)
```

Asynchronously send a record to a topic and invoke the provided callback when the send has been acknowledged.

The `send` is asynchronous and this method will return immediately (except for rare cases described below) once the record has been stored in the buffer of records waiting to be sent. This allows sending many records in parallel without blocking to wait for the response after each one. Can block for the following cases: 1) For the first record being sent to the cluster by this client for the given topic. In this case it will block for up to `max.block.ms` milliseconds if Kafka cluster is unreachable; 2) Allocating a buffer if buffer pool doesn't have any free buffers.

The result of the send is a `RecordMetadata` specifying the partition the record was sent to, the offset it was assigned and the timestamp of the record. If the producer is configured with `acks = 0`, the `RecordMetadata` will have `offset = -1` because the producer does not wait for the acknowledgement from the broker. If `CreateTime` is used by the topic, the timestamp will be the user provided timestamp or the record send time if the user did not specify a timestamp for the record. If `LogAppendTime` is used for the topic, the timestamp will be the Kafka broker local time when the message is appended.

Since the send call is asynchronous it returns a `Future` for the `RecordMetadata` that will be assigned to this record. Invoking `get()` on this future will block until the associated request completes and then return the metadata for the record or throw any exception that occurred while sending the record.

If you want to simulate a simple blocking call you can call the `get()` method immediately:

```
byte[] key = "key".getBytes();
byte[] value = "value".getBytes();
ProducerRecord<byte[],byte[]> record = new ProducerRecord<byte[],byte[]>("my-topic", key, value)
producer.send(record).get();
```

Fully non-blocking usage can make use of the `Callback` parameter to provide a callback that will be invoked when the request is complete.

```
ProducerRecord<byte[],byte[]> record = new ProducerRecord<byte[],byte[]>("the-topic", key, value);
producer.send(myRecord,
    new Callback() {
        public void onCompletion(RecordMetadata metadata, Exception e) {
            if(e != null) {
                e.printStackTrace();
            } else {
                System.out.println("The offset of the record we just sent is: " + metadata.offset());
            }
        }
    });
```

Callbacks for records being sent to the same partition are guaranteed to execute in order. That is, in the following example `callback1` is guaranteed to execute before `callback2`:

```
producer.send(new ProducerRecord<byte[],byte[]>(topic, partition, key1, value1), callback1);
producer.send(new ProducerRecord<byte[],byte[]>(topic, partition, key2, value2), callback2);
```

When used as part of a transaction, it is not necessary to define a callback or check the result of the future in order to detect errors from `send`. If any of the send calls failed with an irrecoverable error, the final `commitTransaction()` call will fail and throw the exception from the last failed send. When this happens, your application should call `abortTransaction()` to reset the state and continue to send data.

Some transactional send errors cannot be resolved with a call to `abortTransaction()`. In particular, if a transactional send finishes with a `ProducerFencedException`, a `OutOfOrderSequenceException`, a `UnsupportedVersionException`, or an `AuthorizationException`, then the only option left is to call `close()`. Fatal errors cause the producer to enter a defunct state in which future API calls will continue to raise the same underlying error wrapped in a new `KafkaException`.

It is a similar picture when idempotence is enabled, but no `transactional.id` has been configured. In this case, `UnsupportedVersionException` and `AuthorizationException` are considered fatal errors. However, `ProducerFencedException` does not need to be handled. Additionally, it is possible to continue sending after receiving an `OutOfOrderSequenceException`, but doing so can result in out of order delivery of pending messages. To ensure proper ordering, you should close the producer and create a new instance.

If the message format of the destination topic is not upgraded to 0.11.0.0, idempotent and transactional produce requests will fail with an `UnsupportedForMessageFormatException` error. If this is encountered during a transaction, it is possible to abort and continue. But note that future sends to the same topic will continue receiving the same exception until the topic is upgraded.

Note that callbacks will generally execute in the I/O thread of the producer and so should be reasonably fast or they will delay the sending of messages from other threads. If you want to execute blocking or computationally expensive callbacks it is recommended to use your own `Executor` in the callback body to parallelize processing.

Specified by:

`send` in interface `Producer<K,V>`

Parameters:

`record` - The record to send

`callback` - A user-supplied callback to execute when the record has been acknowledged by the server (null indicates no callback)

Throws:

`IllegalStateException` - if a `transactional.id` has been configured and no transaction has been started, or when `send` is invoked after producer has been closed.

`InterruptedException` - If the thread is interrupted while blocked

`SerializationException` - If the key or value are not valid objects given the configured serializers

`KafkaException` - If a Kafka related error occurs that does not belong to the public API exceptions.

flush

```
public void flush()
```

Invoking this method makes all buffered records immediately available to send (even if `linger.ms` is greater than 0) and blocks on the completion of the requests associated with these records. The post-condition of `flush()` is that any previously sent record will have completed (e.g. `Future.isDone() == true` and callbacks passed to `send(ProducerRecord, Callback)` have been called). A request is considered completed when it is successfully acknowledged according to the `acks` configuration you have specified or else it results in an error.

Other threads can continue sending records while one thread is blocked waiting for a flush call to complete, however no guarantee is made about the completion of records sent after the flush call begins.

This method can be useful when consuming from some input system and producing into Kafka. The `flush()` call gives a convenient way to ensure all previously sent messages have actually completed.

This example shows how to consume from one Kafka topic and produce to another Kafka topic:

```
for(ConsumerRecord<String, String> record: consumer.poll(100))
    producer.send(new ProducerRecord("my-topic", record.key(), record.value()));
producer.flush();
consumer.commitSync();
```

Note that the above example may drop records if the produce request fails. If we want to ensure that this does not occur we need to set `retries=<large_number>` in our config.

Applications don't need to call this method for transactional producers, since the `commitTransaction()` will flush all buffered records before performing the commit. This ensures that all the `send(ProducerRecord)` calls made since the previous `beginTransaction()` are completed before the commit.

Important: This method must not be called from within the callback provided to `send(ProducerRecord, Callback)`. Invoking `flush()` in this context will result in a `KafkaException` being thrown, as it will cause a deadlock.

Specified by:

`flush` in interface `Producer<K,V>`

Throws:

`InterruptedException` - If the thread is interrupted while blocked

`KafkaException` - If the method is invoked inside a `send(ProducerRecord, Callback)` callback

partitionsFor

```
public List<PartitionInfo> partitionsFor(String topic)
```

Get the partition metadata for the given topic. This can be used for custom partitioning.

Specified by:

`partitionsFor` in interface `Producer<K,V>`

Throws:

`AuthenticationException` - if authentication fails. See the exception for more details

`AuthorizationException` - if not authorized to the specified topic. See the exception for more details

`InterruptedException` - if the thread is interrupted while blocked

`TimeoutException` - if metadata could not be refreshed within `max.block.ms`

`KafkaException` - for all Kafka-related exceptions, including the case where this method is called after producer close

metrics

```
public Map<MetricName, ? extends Metric> metrics()
```

Get the full set of internal metrics maintained by the producer.

Specified by:

`metrics` in interface `Producer<K,V>`

registerMetricForSubscription

```
public void registerMetricForSubscription(KafkaMetric metric)
```

Add the provided application metric for subscription. This metric will be added to this client's metrics that are available for subscription and sent as telemetry data to the broker. The provided metric must map to an OTLP metric data point type in the OpenTelemetry v1 metrics protobuf message types. Specifically, the metric should be one of the following:

- ``Sum``: Monotonic total count meter (Counter). Suitable for metrics like total number of X, e.g., total bytes sent.
- ``Gauge``: Non-monotonic current value meter (UpDownCounter). Suitable for metrics like current value of Y, e.g., current queue count.

Metrics not matching these types are silently ignored. Executing this method for a previously registered metric is a benign operation and results in updating that metrics entry.

Specified by:

`registerMetricForSubscription` in interface `Producer<K,V>`

Parameters:

`metric` - The application metric to register

See Also:

`registerMetricForSubscription(KafkaMetric)`

unregisterMetricFromSubscription

```
public void unregisterMetricFromSubscription(KafkaMetric metric)
```

Remove the provided application metric for subscription. This metric is removed from this client's metrics and will not be available for subscription any longer. Executing this method with a metric that has not been registered is a benign operation and does not result in any action taken (no-op).

Specified by:

`unregisterMetricFromSubscription` in interface `Producer<K,V>`

Parameters:

`metric` - The application metric to remove

See Also:

`unregisterMetricFromSubscription(KafkaMetric)`

clientId

```
public Uuid clientId(Duration timeout)
```

Determines the client's unique client instance ID used for telemetry. This ID is unique to this specific client instance and will not change after it is initially generated. The ID is useful for correlating client operations with telemetry sent to the broker and to its eventual monitoring destinations.

If telemetry is enabled, this will first require a connection to the cluster to generate the unique client instance ID. This method waits up to `timeout` for the producer client to complete the request.

Client telemetry is controlled by the `ProducerConfig.ENABLE_METRICS_PUSH_CONFIG` configuration option.

Specified by:

`clientId` in interface `Producer<K,V>`

Parameters:

`timeout` - The maximum time to wait for producer client to determine its client instance ID. The value must be non-negative. Specifying a timeout of zero means do not wait for the initial request to complete if it hasn't already.

Returns:

The client's assigned instance id used for metrics collection.

Throws:

`InterruptedException` - If the thread is interrupted while blocked.

`KafkaException` - If an unexpected error occurs while trying to determine the client instance ID, though this error does not necessarily imply the producer client is otherwise unusable.

`IllegalArgumentException` - If the `timeout` is negative.

`IllegalStateException` - If telemetry is not enabled ie, config ``enable.metrics.push`` is set to ``false``.

close

```
public void close()
```

Close this producer. This method blocks until all previously sent requests complete. This method is equivalent to `close(Long.MAX_VALUE, TimeUnit.MILLISECONDS)`.

If `close()` is called from `Callback`, a warning message will be logged and `close(o, TimeUnit.MILLISECONDS)` will be called instead. We do this because the sender thread would otherwise try to join itself and block forever.

Specified by:

`close` in interface `AutoCloseable`

Specified by:

`close` in interface `Closeable`

Specified by:

`close` in interface `Producer<K,V>`

Throws:

`InterruptedException` - If the thread is interrupted while blocked.

`KafkaException` - If an unexpected error occurs while trying to close the client, this error should be treated as fatal and indicate the client is no longer usable.

close

```
public void close(Duration timeout)
```

This method waits up to `timeout` for the producer to complete the sending of all incomplete requests.

If the producer is unable to complete all requests before the timeout expires, this method will fail any unsent and unacknowledged records immediately. It will also abort the ongoing transaction if it's not already completing.

If invoked from within a `Callback` this method will not block and will be equivalent to `close(Duration.ofMillis(0))`. This is done since no further sending will happen while blocking the I/O thread of the producer.

Specified by:

`close` in interface `Producer<K,V>`

Parameters:

`timeout` - The maximum time to wait for producer to complete any pending requests. The value should be non-negative. Specifying a timeout of zero means do not wait for pending send requests to complete.

Throws:

`InterruptedException` - If the thread is interrupted while blocked.

`KafkaException` - If an unexpected error occurs while trying to close the client, this error should be treated as fatal and indicate the client is no longer usable.

`IllegalArgumentException` - If the `timeout` is negative.